Adaptation-Aware Model Selection for Few-Shot Digital Twin Instantiation under Deployment Limits

ARTICLE INFO

Keywords:
digital twin
few-shot adaptation
model instantiation
architecture selection
model deployment limits
heterogeneous computing centers

ABSTRACT

As an important modeling tool for managing computing centers, digital twin (DT) platforms reuse pre-trained DT models from existing centers to reduce the cost of DT instantiation in large-scale computing networks. However, new or reconfigured centers may differ substantially from existing centers in resource configurations and workload characteristics while providing only limited target-specific data for DT instantiation. Although existing few-shot adaptation methods can reuse pre-trained models with limited target data, they typically update model parameters within a predefined architecture. For heterogeneous target centers, however, the source architecture may be suboptimal, and the instantiated model must also satisfy per-model operation-count and parameter-count limits. To address this challenge, we propose Model Selection and Adaptation for Digital Twin Instantiation (MSA-DTI), an adaptation-aware model-selection algorithm for few-shot DT instantiation under deployment limits. MSA-DTI first constructs a candidate model bank by pre-training models with diverse architectures on historical workload data, filters out infeasible candidates according to the deployment limits, and adapts the remaining candidates using limited target samples. To make model selection more conservative with limited target data, it compares the best-performing alternative with a predefined reference model after adaptation and requires a preset relative reduction in validation loss for replacement. Comparative experiments show that MSA-DTI achieves the lowest mean squared error (MSE) among the evaluated methods, reducing MSE by 14.60% on average across paired cases relative to the pre-training-and-fine-tuning baseline while satisfying the deployment limits in all main cases.

1. Introduction

As an important modeling tool for managing computing centers, a shared digital twin (DT) platform maintains center-specific models for monitoring, prediction, analysis, and decision making [1–3, 14]. In large-scale computing networks, pre-trained DT models can be reused and adapted instead of constructing each target model from scratch, reducing DT-instantiation cost [4, 5]. Each pre-trained model comprises an architecture and source-trained parameters. The architecture determines the model structure and deployment-related complexity, whereas the parameters provide the initialization for target adaptation. However, a new or reconfigured center may differ from existing centers in resource configurations, monitored-variable availability, workload patterns, and data distributions while only a small support set may be available for target adaptation. These differences can make the original architecture suboptimal for the target center. In addition, large-scale DT platforms impose per-model resource budgets, represented here by limits on estimated operation count per inference and parameter count. Target-center instantiation therefore requires selecting a suitable architecture and adapting its parameters from this small support set while satisfying both deployment limits. Figure 1 illustrates this setting.

Existing few-shot adaptation methods reuse pre-trained models [6, 7] and have also been applied to DT modeling [5]. However, these methods typically retain the original architecture, leaving unresolved whether it remains suitable for a heterogeneous target center and satisfies the deployment limits. Deployment-aware architecture-selection methods consider model performance together with deployment cost [8–10], but do not directly address few-shot adaptation of reusable pre-trained models. Methods that jointly consider architecture decisions and few-shot adaptation [11–13] typically rely on target-side architecture or adaptation-structure search. The remaining challenge is therefore to select a suitable reusable pre-trained DT model under per-model deployment limits and adapt its parameters from a small target support set.

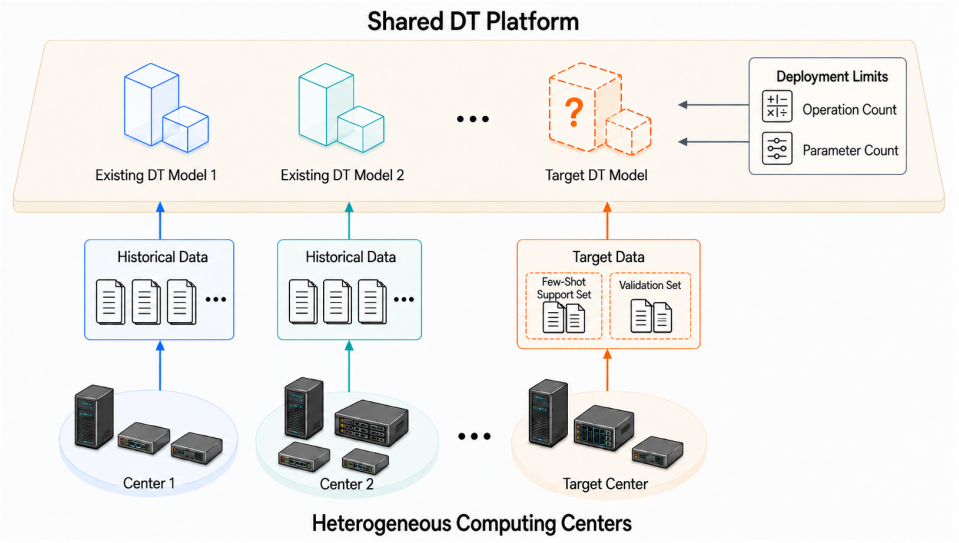


Figure 1: System setting for digital twin model instantiation with few-shot target adaptation and deployment limits.

To address this challenge, we propose Model Selection and Adaptation for Digital Twin Instantiation (MSA-DTI), an adaptation-aware model-selection algorithm for few-shot DT instantiation under deployment limits. MSA-DTI first constructs a reusable candidate bank of source-trained models with diverse architectures. For a target center, it filters candidates that violate the deployment limits and adapts the remaining feasible candidates under a common adaptation budget. It then compares the adapted candidates using validation performance and retains a predefined reference model unless the best-performing alternative achieves a sufficient reduction in validation loss. Thus, model selection is based on post-adaptation performance without target-side architecture or adaptation-structure search.

The main contributions are summarized as follows:

- **Deployment-constrained few-shot DT instantiation formulation.** We formulate DT instantiation as jointly selecting an architecture from reusable source-trained models and adapting the corresponding model parameters from a small target support set, subject to hard per-model operation-count and parameter-count limits.
- **Feasibility-first adaptation-aware model selection.** MSA-DTI filters out infeasible candidates before target adaptation and compares the remaining candidates only after adapting them using the same target support set and update budget.
- **Empirically calibrated reference-model replacement rule.** We introduce a relative-improvement threshold calibrated on disjoint development data, requiring an adapted alternative to achieve the required relative reduction in validation loss before replacing the predefined reference model.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 defines the system model and problem. Section 4 presents MSA-DTI. Section 5 reports the experiments, and Section 6 concludes the paper.

2. Related Work

2.1. Digital Twin Modeling and Platform Deployment

In communication and computing systems, digital twins represent system states and services while supporting model updates [4, 15–17]. At the deployment level, related work considers service orchestration and computation offloading under resource constraints [18, 19]. However, DT modeling studies typically assume a predefined model structure, whereas platform-level research focuses on resource decisions. Neither line of work directly addresses architecture selection together with few-shot target adaptation under deployment limits.

2.2. Limited-Data Model Adaptation

Limited-data methods leverage source or prior knowledge for target-model adaptation [6, 7, 20]. Recent approaches improve cross-task and cross-domain adaptation through context modeling and transferable representations [21–24]. Limited-data learning has also been applied to DT construction and workload prediction [5, 25]. Most parameter-adaptation methods operate with a fixed architecture. For heterogeneous target centers, a fixed architecture may be unsuitable, motivating selection among reusable source-trained models.

Model selection introduces a related challenge because validation-based performance estimates may yield uncertain candidate rankings [33, 34]. This motivates a conservative decision rule when adapted reusable models are compared on target validation data.

2.3. Deployment-Aware Architecture Selection

Deployment-aware neural architecture search evaluates model performance together with deployment cost [8–10]. Few-shot, zero-shot, and training-free NAS variants reduce the cost of architecture evaluation during search [26–28]; in Ref. [26], "few-shot" denotes evaluation with several sub-supernets rather than adaptation from a small target support set.

Among meta-learning-based NAS methods, T-NAS learns transferable architecture knowledge for new-task adaptation [12]. H-Meta-NAS further incorporates hardware constraints into task-adaptive search [11]. Neural fine-tuning search instead starts from a pre-trained backbone and searches over adaptation structures, including which components to fine-tune or augment [13]. Together, these methods extend task adaptation beyond parameter updates to architecture or adaptation-structure decisions.

In contrast, MSA-DTI operates on a finite bank of separately source-trained reusable models and performs deployment-constrained selection based on post-adaptation performance, without target-stage search for new architectures or adaptation structures.

3. System Model and Problem Formulation

3.1. System Setting and Target Model

We consider a shared DT platform that maintains center-specific models for heterogeneous computing centers.

Let c denote a new or reconfigured target center. To enable model reuse across centers with different monitored-variable availability, all observations follow a common metric schema, with unavailable variables represented by masks rather than omitted. Let $\mathcal{X}$ and $\mathcal{Y}$ denote the common input and output spaces, respectively. Let $\mathcal{A}$ denote the architecture space. For $a \in \mathcal{A}$, let $\Theta(a)$ denote its parameter space and $\theta \in \Theta(a)$ a parameter vector. A model is represented as

$$f_{a,\theta} : \mathcal{X} \to \mathcal{Y}, \qquad a \in \mathcal{A},\ \theta \in \Theta(a), \tag{1}$$

The DT model instantiated for center c is represented as

$$\mathrm{DT}_c = \left(a_c, \theta_c\right), \qquad a_c \in \mathcal{A},\ \theta_c \in \Theta(a_c), \tag{2}$$

where a_c and θ_c denote the target architecture and target-specific parameters, respectively.

During instantiation, the available target data are represented by a support set and a validation set:

$$\begin{aligned} \mathcal{D}_c^{\mathrm{sup}} &= \left\{ (x_{c,i}^{\mathrm{sup}}, y_{c,i}^{\mathrm{sup}}) \right\}_{i=1}^{K_c}, \\ \mathcal{D}_c^{\mathrm{val}} &= \left\{ (x_{c,j}^{\mathrm{val}}, y_{c,j}^{\mathrm{val}}) \right\}_{j=1}^{V_c}, \\ \mathcal{D}_c^{\mathrm{tar}} &= \mathcal{D}_c^{\mathrm{sup}} \cup \mathcal{D}_c^{\mathrm{val}}, \qquad \mathcal{D}_c^{\mathrm{sup}} \cap \mathcal{D}_c^{\mathrm{val}} = \varnothing. \end{aligned} \tag{3}$$

In these sets, $(x_{c,i}^{\mathrm{sup}}, y_{c,i}^{\mathrm{sup}})$ and $(x_{c,j}^{\mathrm{val}}, y_{c,j}^{\mathrm{val}})$ denote the ith support and jth validation input-output pairs, respectively. Their union $\mathcal{D}_c^{\mathrm{tar}}$ denotes the target data available during instantiation. K_c denotes the number of target support samples used for parameter adaptation, whereas V_c denotes the number of validation samples used for post-adaptation candidate comparison. The present problem concerns determining (a_c, θ_c) from these target data; later model updates are outside the scope of this study.

3.2. Reusable Models and Deployment Limits

Let the finite set of N reusable source-trained models be

$$\mathcal{Q} = \{q_n\}_{n=1}^{N}, \qquad q_n = (a_n, \theta_n^0), \tag{4}$$

where q_n denotes the nth reusable source-trained model, $a_n \in \mathcal{A}$ denotes its architecture, and $\theta_n^0 \in \Theta(a_n)$ denotes its source-trained initialization. The architectures represented in $\mathcal{Q}$ form the set $\mathcal{A}_{\mathcal{Q}} = \{a_n\}_{n=1}^{N} \subseteq \mathcal{A}$.

For target center c, let B_c^F and B_c^P denote the operation-count and parameter-count limits, respectively. For $a \in \mathcal{A}_{\mathcal{Q}}$, let $F_c(a)$ denote the estimated operation count of a batch-size-one forward pass under a fixed counting rule, and let $P_c(a)$ denote the exact number of trainable parameters. In this formulation, parameter adaptation changes only parameter values while keeping the architecture fixed; therefore, $F_c(a)$ and $P_c(a)$ remain unchanged.

For center c, the deployment-feasible reusable model set is

$$\mathcal{Q}_c^{\mathrm{fea}} = \left\{ q_n \in \mathcal{Q} \;\middle|\; F_c(a_n) \le B_c^F,\ P_c(a_n) \le B_c^P \right\}. \tag{5}$$

Both deployment limits are modeled as hard constraints rather than penalty terms.

3.3. DT Model Instantiation Problem

Let $\mathbb{P}_c$ denote the target-center data distribution over $\mathcal{X} \times \mathcal{Y}$, and let $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_{\ge 0}$ denote the task loss function. The target risk is defined as

$$R_c(a, \theta) = \mathbb{E}_{(x,y)\sim\mathbb{P}_c}\left[\ell(f_{a,\theta}(x), y)\right]. \tag{6}$$

Let $\mathcal{U}_c$ denote a prescribed target-adaptation operator. For each reusable model $q_n = (a_n, \theta_n^0)$, $\mathcal{U}_c(q_n, \mathcal{D}_c^{\mathrm{sup}}) \in \Theta(a_n)$ denotes the target-specific parameters obtained by adapting q_n using the target support set. An ideal reusable-model decision for center c is defined as

$$\begin{aligned} n_c^\dagger \in \underset{n\in\{1,\dots,N\}}{\arg\min} \quad & R_c\left(a_n, \mathcal{U}_c(q_n, \mathcal{D}_c^{\mathrm{sup}})\right) \\ \text{s.t.} \quad & F_c(a_n) \le B_c^F, \\ & P_c(a_n) \le B_c^P. \end{aligned} \tag{7}$$

Here, $n_c^\dagger$ denotes the index of an ideal reusable-model choice for target center c. The corresponding ideal DT model is $\left(a_c^\dagger, \theta_c^\dagger\right) = \left(a_{n_c^\dagger}, \mathcal{U}_c(q_{n_c^\dagger}, \mathcal{D}_c^{\mathrm{sup}})\right)$. During instantiation, however, $\mathbb{P}_c$, and hence the target risk R_c, is unavailable; only the support and validation sets in Eq. (3) are available for target adaptation and candidate comparison.

4. The Proposed MSA-DTI Algorithm

4.1. Overview

Figure 2 summarizes the three-stage MSA-DTI workflow for addressing the instantiation problem in Eq. (7): candidate bank construction, dual-limit candidate filtering, and common-budget adaptation followed by reference-based selection. The first stage combines architecture screening and source pretraining to construct a reusable candidate bank and designates one candidate as the reference. For each target center, candidates that violate the deployment limits are removed before adaptation. The remaining candidates are adapted using the same target support set and update budget, after which their post-adaptation performance is compared on a common validation set. A calibrated relative margin then specifies the relative reduction in validation loss that the best-performing alternative must achieve to replace the adapted reference.

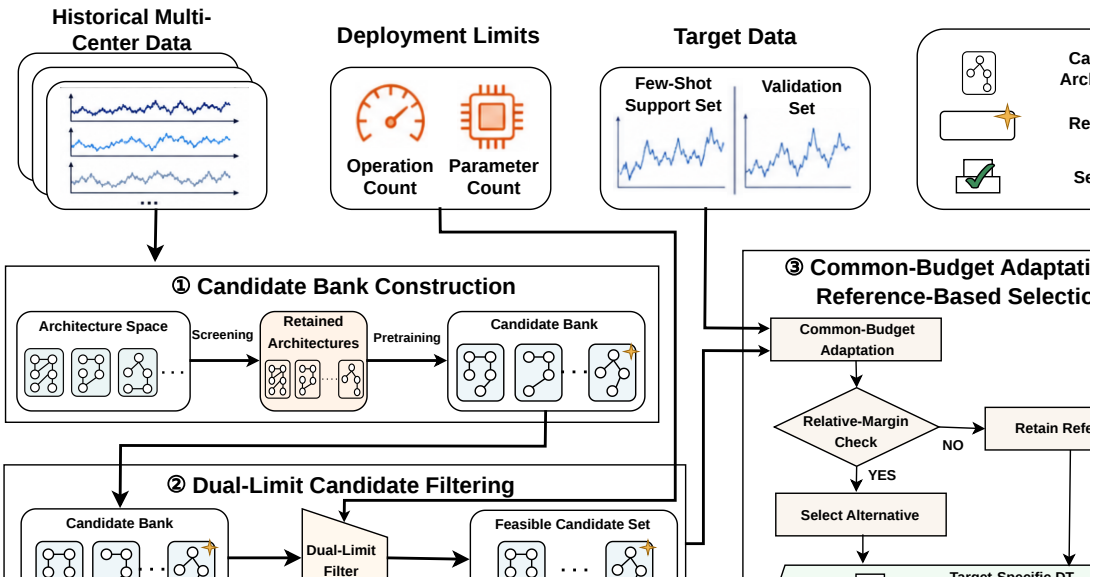


Figure 2: Overall workflow of MSA-DTI: candidate bank construction, dual-limit candidate filtering, and common-budget adaptation with reference-based selection.

4.2. Candidate Bank Construction

Architecture screening uses a dedicated screening pool that is disjoint from the data used for margin calibration and final target evaluation. A reference architecture $a^{\mathrm{ref}} \in \mathcal{A}$ is fixed before screening and retained regardless of the screening outcome. Each screening case contains three subsets: support for adaptation, validation for selecting an alternative architecture, and check for screening evidence. For each screening case, the reference architecture must satisfy the assigned deployment limits, and only feasible alternative architectures are included in the comparison. During screening, the reference and each feasible alternative are adapted from fixed source-trained screening initializations on the support subset before validation and check losses are evaluated.

For each alternative architecture, two evidence counts are accumulated across the screening cases. A *check-oracle win* is assigned to the feasible alternative architecture with the lowest check-set loss if its loss is lower than that of the adapted reference. A *validation-selected positive check win* is assigned to the feasible alternative architecture selected on the validation subset if its check-set loss is lower than that of the adapted reference. The architecture is retained if at least one of the two counts reaches a preset threshold.

Let $\mathcal{D}_{\mathrm{src}}$ denote the combined historical data from the existing centers. For each retained architecture a_q, source pretraining on $\mathcal{D}_{\mathrm{src}}$ produces an initialization θ_q^0, forming a candidate $q = (a_q, \theta_q^0) \in \mathcal{Q}$. For a retained architecture with multiple source-trained initializations, each initialization defines a distinct candidate. One candidate associated with the reference architecture a^{ref} is designated as q^{ref}. Figure 3 summarizes the retained architectures and resulting candidate bank. The candidate bank and reference candidate are fixed before margin calibration and final target evaluation.

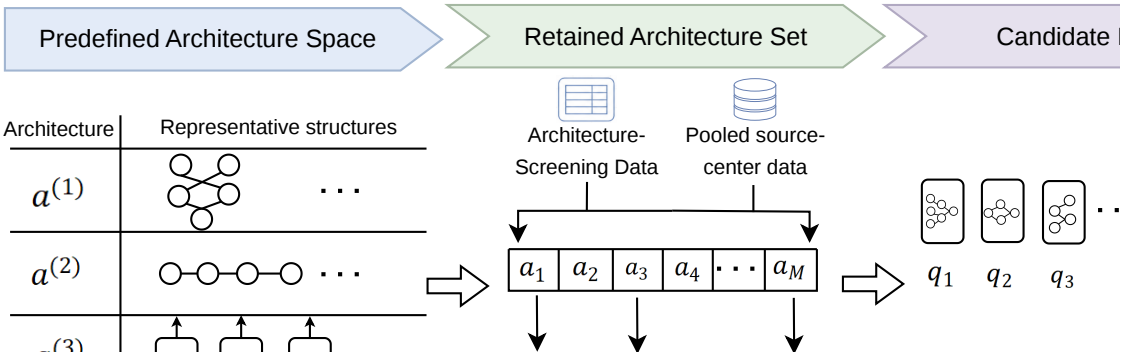


Figure 3: Candidate bank construction and reference candidate designation.

4.3. Dual-Limit Candidate Filtering

For target center c, MSA-DTI computes $\mathcal{Q}_c^{\mathrm{fea}}$ using Eq. (5) and restricts target adaptation to the candidates in this set. Because the subsequent reference-based selection requires a deployment-feasible reference for comparison and fallback, MSA-DTI proceeds only if $q^{\mathrm{ref}} \in \mathcal{Q}_c^{\mathrm{fea}}$; otherwise, it terminates without DT instantiation under the current deployment limits.

4.4. Common-Budget Adaptation and Reference-Based Selection

For target-center instantiation, the support set $\mathcal{D}_c^{\mathrm{sup}}$ is used for parameter adaptation, whereas the validation set $\mathcal{D}_c^{\mathrm{val}}$ is used for post-adaptation candidate comparison. For a dataset $\mathcal{D}$, let

$$\mathcal{L}(\mathcal{D}; a, \theta) = \frac{1}{|\mathcal{D}|} \sum_{(x,y)\in\mathcal{D}} \ell\left(f_{a,\theta}(x), y\right)$$

denote the empirical task loss.

For each feasible candidate $q = (a_q, \theta_q^0)$, let $\theta_{c,q}^{(t)}$ denote its parameters after t target-side adaptation steps. With a common update budget of T steps and a learning rate η shared by all feasible candidates, the adaptation is

$$\begin{aligned} \theta_{c,q}^{(0)} &= \theta_q^0, \\ \theta_{c,q}^{(t+1)} &= \theta_{c,q}^{(t)} - \eta \nabla_\theta \mathcal{L}\left(\mathcal{D}_c^{\mathrm{sup}}; a_q, \theta_{c,q}^{(t)}\right), \\ & \qquad t = 0, \ldots, T-1. \end{aligned} \tag{8}$$

Equation (8) gives the gradient-based realization of the target-adaptation operator $\mathcal{U}_c$ introduced in Section 3.3, with $\theta_{c,q}^{(T)}$ as its output. After T adaptation steps, let $J_c(q)$ denote the post-adaptation validation score of candidate q:

$$J_c(q) = \mathcal{L}\left(\mathcal{D}_c^{\mathrm{val}}; a_q, \theta_{c,q}^{(T)}\right). \tag{9}$$

Because validation-based candidate rankings may be uncertain, MSA-DTI uses a relative margin so that a marginal post-adaptation loss reduction does not by itself trigger replacement of the reference candidate.

Let $\mathcal{Q}_c^{\mathrm{alt}} = \mathcal{Q}_c^{\mathrm{fea}} \setminus \{q^{\mathrm{ref}}\}$ denote the feasible alternatives, and let q_c^* denote the candidate selected by MSA-DTI. If $\mathcal{Q}_c^{\mathrm{alt}}$ is empty, MSA-DTI sets $q_c^* = q^{\mathrm{ref}}$ and skips alternative selection. Otherwise, let $\widetilde{q}_c \in \arg\min_{q\in\mathcal{Q}_c^{\mathrm{alt}}} J_c(q)$ denote an alternative with the lowest validation loss. Let $\tau \in [0, 1)$ denote the relative margin. The selected candidate is then

$$q_c^* = \begin{cases} \widetilde{q}_c, & J_c(\widetilde{q}_c) \le (1-\tau) J_c(q^{\mathrm{ref}}), \\ q^{\mathrm{ref}}, & \text{otherwise.} \end{cases} \tag{10}$$

A larger τ requires a larger validation-loss reduction for an alternative to be selected over the reference. The margin affects only model selection and does not change the adapted parameters.

The selected candidate q_c^* provides the architecture $a_{q_c^*}$, and its adapted parameters $\theta_{c,q_c^*}^{(T)}$ define the target-specific DT model in Eq. (2). Figure 4 illustrates the adaptation and selection stages.

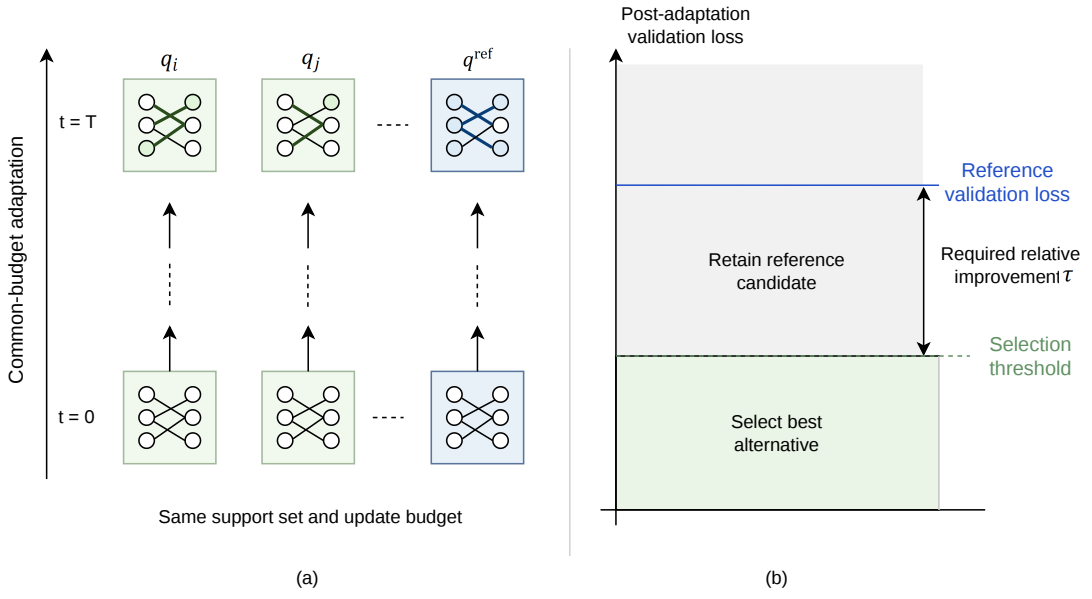


Figure 4: Common-budget adaptation and reference-based selection.

Algorithm 1 summarizes the target-side procedure.

Algorithm 1 Target-Side MSA-DTI for One Center

Require: Candidate bank $\mathcal{Q}$, reference candidate q^{ref}, support set $\mathcal{D}_c^{\text{sup}}$, validation set $\mathcal{D}_c^{\text{val}}$, deployment limits B_c^F and B_c^P, update budget T, learning rate η, and relative margin τ

Ensure: Target-specific DT model DT_c, if the reference candidate is deployment-feasible

Compute $\mathcal{Q}_c^{\text{fea}}$ using Eq. (5)
if $q^{\text{ref}} \notin \mathcal{Q}_c^{\text{fea}}$ **then**
    terminate without DT instantiation
end if
for all $q \in \mathcal{Q}_c^{\text{fea}}$ **do**
    Adapt q using Eq. (8)
    Compute $J_c(q)$ using Eq. (9)
end for
$\mathcal{Q}_c^{\text{alt}} \leftarrow \mathcal{Q}_c^{\text{fea}} \setminus \{q^{\text{ref}}\}$
if $\mathcal{Q}_c^{\text{alt}} = \emptyset$ **then**
    $q_c^* \leftarrow q^{\text{ref}}$
else
    Choose $\widetilde{q}_c \in \arg\min_{q \in \mathcal{Q}_c^{\text{alt}}} J_c(q)$
    Select q_c^* using Eq. (10)
end if
$\text{DT}_c \leftarrow \left(a_{q_c^*}, \theta_{c,q_c^*}^{(T)}\right)$
return DT_c

4.5. Relative-Margin Calibration

The margin τ is calibrated on a margin-calibration pool that is disjoint from the architecture-screening cases and final target evaluation. Each calibration case contains support, validation, and check sets. The support and validation sets retain the roles defined in Section 4.4, whereas the check set provides independent evidence for evaluating the resulting selection during calibration. For a given margin, the harmful-selection rate is the fraction of all calibration

cases in which an alternative is selected and its check-set loss exceeds that of the adapted reference. The paired-initialization control uses a second source-trained initialization of the reference architecture and provides a same-architecture control for source-initialization effects.

Let $\mathcal{T}$ be a preset margin set. For $\tau \in \mathcal{T}$, let $\widehat{R}_{\mathrm{harm}}(\tau)$ denote the observed harmful-selection rate. Let $\widehat{G}_{\mathrm{ref}}(\tau)$ and $\widehat{G}_{\mathrm{pair}}(\tau)$ denote the mean relative check-loss gains with respect to the adapted reference and paired-initialization control, respectively, where each gain is the relative reduction in check-set loss of the selected model with respect to the corresponding control. Their lower confidence bounds are $\underline{G}_{\mathrm{ref}}(\tau)$ and $\underline{G}_{\mathrm{pair}}(\tau)$. Given a harmful-selection-rate threshold ρ and mean-gain thresholds γ_{ref} and γ_{pair}, the eligible margins are

$$\begin{aligned}\mathcal{T}_{\mathrm{elig}} = \big\{\tau \in \mathcal{T} \;\big|\; & \widehat{R}_{\mathrm{harm}}(\tau) \le \rho, \\ & \widehat{G}_{\mathrm{ref}}(\tau) \ge \gamma_{\mathrm{ref}}, \quad \underline{G}_{\mathrm{ref}}(\tau) > 0, \\ & \widehat{G}_{\mathrm{pair}}(\tau) \ge \gamma_{\mathrm{pair}}, \quad \underline{G}_{\mathrm{pair}}(\tau) > 0\big\}.\end{aligned} \tag{11}$$

When $\mathcal{T}_{\mathrm{elig}}$ is nonempty, the calibrated margin is the least restrictive eligible margin,

$$\tau^* = \min \mathcal{T}_{\mathrm{elig}},$$

and is fixed before final target evaluation.

If $\mathcal{T}_{\mathrm{elig}} = \varnothing$, MSA-DTI retains the adapted reference. Because the statistics in Eq. (11) are estimated from the calibration pool, the calibrated margin depends on the calibration domain.

4.6. Analytical Properties

By construction, any DT model instantiated by MSA-DTI is selected from $\mathcal{Q}_c^{\mathrm{fea}}$ and therefore satisfies both deployment limits. Moreover, whenever an alternative is selected and $J_c(q^{\mathrm{ref}}) > 0$, Eq. (10) guarantees a relative validation-loss reduction of at least τ with respect to the reference.

The target-stage computational complexity is

$$O\left(|\mathcal{Q}| + \sum_{q \in \mathcal{Q}_c^{\mathrm{fea}}} \left(T C_q^{\mathrm{grad}} + C_q^{\mathrm{val}}\right)\right),$$

where C_q^{grad} and C_q^{val} denote the costs of one target update and validation evaluation for candidate q, respectively, while the $|\mathcal{Q}|$ term accounts for deployment filtering.

5. Experimental Evaluation

We evaluate prediction accuracy, selection reliability, deployment feasibility, target-side cost, and robustness across target conditions.

5.1. Experimental Setup

5.1.1. Benchmark and Data Protocol

We use a controlled synthetic benchmark for heterogeneous multi-center workloads. Center types A–C are evaluation strata with progressively sparser and noisier monitoring and distinct workload-dynamics ranges; deployment-limit tiers are separate platform-side settings.

The main study uses 20 source centers and 20 held-out target centers. Prediction horizons $H \in \{1, 4\}$ and support sizes $K \in \{10, 20\}$ give four cases per target center and 80 held-out cases in total. Here, K denotes the number of support windows used for target adaptation.

For each case, the raw timeline is partitioned chronologically into disjoint support, validation, check, and test segments before window generation, and windows are constructed separately within each segment. Depending on H and K, the validation, check, and test sets contain 1342–1351, 621–628, and 621–627 windows, respectively. Across the architecture-screening, margin-calibration, and diagnostic pools, check sets provide the corresponding development evidence, whereas held-out target test sets are reserved for final evaluation after model selection. The source, architecture-screening, margin-calibration, diagnostic, and held-out target pools use disjoint center identities.

5.1.2. Candidate Bank, Reference, and Calibration

The initial architecture space contains 66 configurations spanning multilayer perceptron (MLP), temporal convolutional network (TCN), and gated recurrent unit (GRU) architectures. A57, a one-layer GRU with 32 hidden units, is fixed as the reference before screening.

An alternative configuration is retained if either of the two screening evidence counts defined in Section 4.2 is at least two. The screening retains six configurations corresponding to five executable architectures. Together with a second source-trained initialization of A57 used as the paired-initialization control, the resulting source-trained records form a seven-candidate bank.

Applying Eq. (11) to the preset margin grid on the main calibration pool yields the smallest eligible margin $\tau = 0.10$, which is fixed before held-out evaluation. Table 1 summarizes the experimental configuration.

Table 1: Main experimental configuration.

Category	Item	Setting
Data	Centers; observations; input length	Source / held-out: 20/20; 3000 per center; $L = 96$
	Target cases and split sizes	80 cases; $H \in \{1, 4\}$; support $K \in \{10, 20\}$; validation / check / test windows: 1342–1351 / 621–628 / 621–627
Bank	Initial / retained / executable / candidates	66/6/5/7
	Reference and screening	A57: one-layer GRU-32, fixed before screening; alternative retained if either screening evidence count in Section 4.2 is ≥ 2
	Source-model records	Reference pooled-source initialization; five alternative source-trained initializations; one additional A57 paired-initialization control
Training	Source models	Adam / MSE / 50 epochs; learning rate 10^{-3}; batch size 64
	Target adaptation	SGD / MSE / 50 updates; learning rate 0.01; gradient-norm limit 1.0; full support set
Selection	Grid and eligibility	$\{0.05, 0.075, 0.10, 0.125, 0.15, 0.20\}$; harmful-selection rate $\leq 5\%$; both mean relative gains $\geq 3\%$; both 95% lower confidence bounds > 0
	Calibrated margin	$\tau = 0.10$
Limits	Operation-count limits	Tight / medium / loose: $1.5\times10^6/5\times10^6/2\times10^7$
	Parameter-count limits	Tight / medium / loose: $3\times10^4/10^5/5\times10^5$
Statistics	Confidence intervals	Center-cluster bootstrap; 4000 repetitions

5.1.3. Task, Baselines, and Metrics

We use workload prediction as the evaluation task because it supports proactive resource management in computing systems [29–31]. Mean squared error (MSE) is the task loss. We also report mean absolute error (MAE), Worst-10% error, conditional value at risk at the 90th percentile (CVaR90), target-side time, and limit satisfaction.

For each case, Worst-10% error is the mean of the largest 10% of window-level MSE values on the test set. CVaR90 is the mean case-level MSE over the 10% of target cases with the largest MSE.

For held-out reporting, a selected alternative is labeled harmful when its test MSE is higher than that of the adapted reference. Relative gains are computed for each paired case and then averaged. The 95% confidence intervals use a center-cluster bootstrap with 4000 repetitions, resampling target centers while keeping their four cases together.

We compare MSA-DTI with pre-training and fine-tuning (PT+FT), MeDeT-based adaptation, random initialization, few-shot NAS, zero-shot NAS, zero-shot NAS with fine-tuning, and task-matched H-Meta-NAS-based adaptation [11]. PT+FT is the main controlled baseline because it shares the source-trained reference and target adaptation procedure with MSA-DTI. The remaining baselines provide method-level comparisons while retaining their method-specific initialization or search procedures.

As the closest evaluated architecture-adaptation baseline, H-Meta-NAS-based adaptation uses source-learned meta-initializations and validation-only target search after deployment filtering. Its meta-initializations and search settings are fixed before held-out evaluation, and the main comparison uses a budget-matched 50-update SGD/MSE setting. All methods use the same target cases, chronological splits, deployment limits, and architecture definitions.

5.1.4. Runtime Protocol

All methods are implemented in Python and PyTorch and run on an NVIDIA RTX 3060 Laptop GPU. Target-side time includes model creation, initialization loading, target-side search when applicable, target adaptation, validation evaluation, and final selection, but excludes offline preparation and test evaluation. Each reported time is the mean over five runs.

5.2. Overall Performance and Selection Reliability

Figure 5 compares predictive performance on the held-out target cases. All selected models satisfy both deployment limits.



Figure 5: Predictive performance on the held-out target cases: (a) MAE, (b) MSE, (c) Worst-10% error, and (d) CVaR90 (lower is better).

MSA-DTI gives the lowest values for all four error measures. Compared with PT+FT, the strongest baseline, MSA-DTI reduces MAE by 8.55%, MSE by 14.60%, and Worst-10% error by 13.76% on average across paired cases. The 95% confidence interval for the paired MSE reduction is [9.41%, 20.14%]. Paired MSE comparisons with the remaining baselines also favor MSA-DTI, with each 95% confidence interval lying above zero.

Figure 6 compares MSA-DTI with PT+FT at the case level.

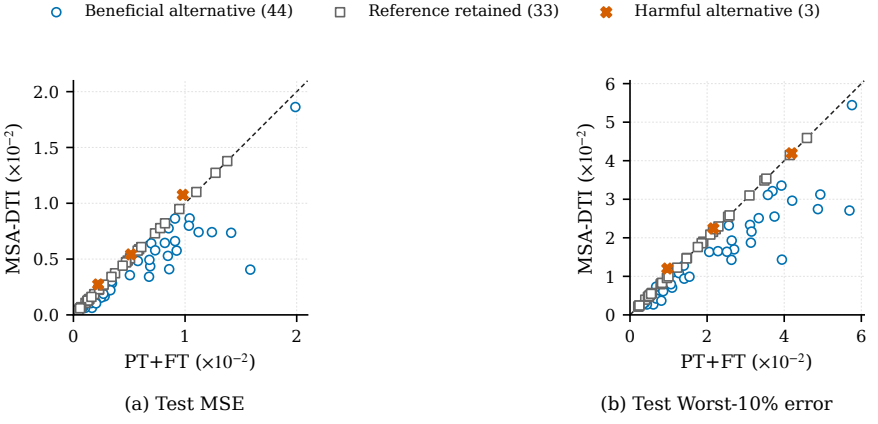


Figure 6: Held-out paired comparison of MSA-DTI and PT+FT for (a) test MSE and (b) test Worst-10% error; beneficial and harmful alternative labels are defined by test MSE, and points below the diagonal favor MSA-DTI.

MSA-DTI retains the reference in 33 cases and selects an alternative in 47 cases. Of the 47 alternative selections, 44 have lower test MSE than the adapted reference and three are harmful, corresponding to 3.75% of all held-out cases. The 95% confidence interval for the all-case harmful-selection rate is [0.00%, 7.50%], so it does not establish a population harmful-selection rate below the 5% calibration criterion.

Because the native NAS baselines use different target optimizers and losses, we additionally evaluate an optimizer-matched diagnostic in which the included methods use SGD, MSE, and 50 target updates per adapted model. In this diagnostic, MSA-DTI achieves the lowest MSE, Worst-10% error, and CVaR90 among the included methods. Table 2 reports these results.

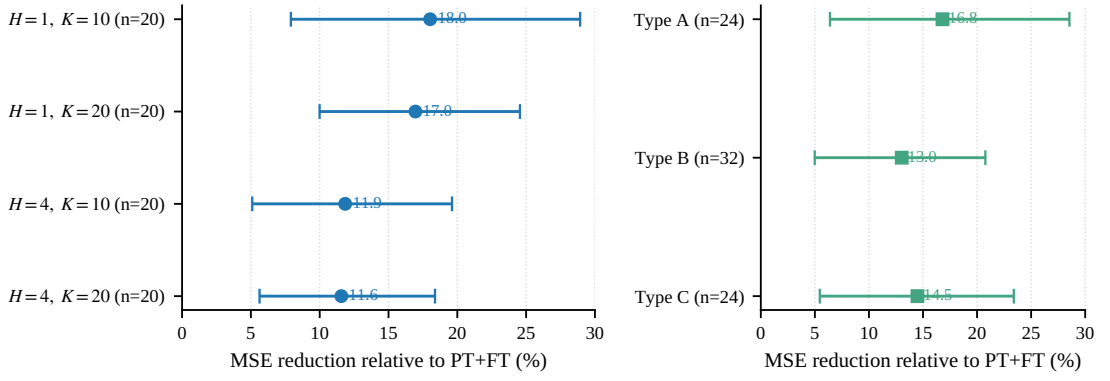
Table 2: Optimizer-matched diagnostic comparison.

Method	MSE↓	Worst-10%↓	CVaR90↓	Adapted models per case
PT+FT	0.004497	0.017144	0.011878	1.00
Few-shot NAS	0.006715	0.025246	0.019653	12.00
Zero-shot NAS + fine-tuning	0.007402	0.028099	0.020402	12.00
MSA-DTI	**0.003809**	**0.015094**	**0.008696**	6.10

All methods use the same independent diagnostic cases, hard deployment limits, SGD optimizer, MSE loss, and 50 target updates per adapted model. The source initializations and candidate-selection rules remain method-specific.

5.3. Robustness and Sensitivity Analysis

Figure 7 reports paired MSE reductions relative to PT+FT. Positive values favor MSA-DTI.



(a) Horizon and support size

(b) Target center type

Figure 7: Paired MSE reduction relative to PT+FT across (a) horizon/support settings and (b) target-center types; error bars show 95% confidence intervals.

The reduction ranges from 11.57% to 18.03% across the four horizon and support-size settings and from 13.03% to 16.80% across the three center types. All seven confidence intervals remain above zero.

On a separate robustness pool, the fixed candidate architectures retrained under three source-training seeds yield paired MSE reductions ranging from 10.52% to 13.21%. All three 95% confidence intervals remain above zero.

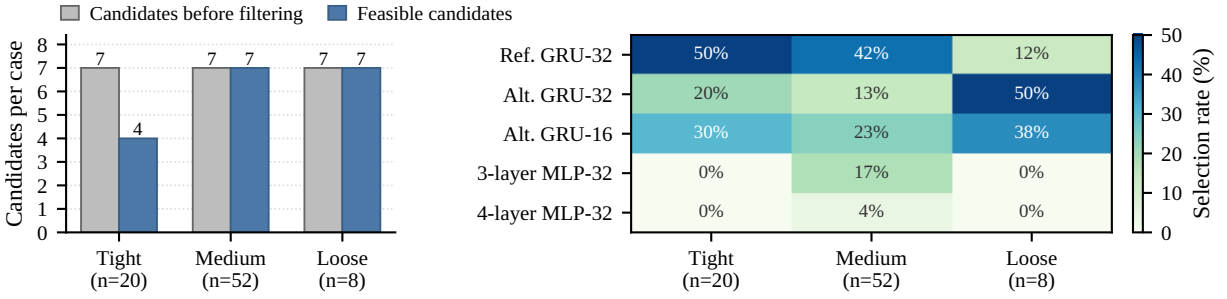
We additionally apply the full 66-configuration screening procedure to two new 20-center pools. The screening procedure retains six configurations for the main-study pool and zero and four configurations for the two independent pools, respectively; the fixed A57 reference candidate remains available for downstream fallback regardless of the screening outcome. The two independent retained sets have Jaccard similarities of 0 and 0.429 to the main-study retained set. A separate screening-stability diagnostic evaluates the two nonempty retained sets on a fourth independent 20-center pool while keeping the downstream procedure unchanged. The 95% confidence intervals for the paired check-MSE gains relative to the reference include zero for both nonempty sets.

The retained-set variation indicates sample sensitivity in candidate discovery. To examine how this variability propagates through the complete instantiation procedure, we then construct the corresponding source-trained banks, apply the same margin-calibration rule to each bank, and evaluate the resulting pipelines on a new 20-center held-out pool. The banks derived from the main-study screening pool and the nonempty independent screening pool achieve mean paired MSE reductions of 10.03% (95% CI [4.60%, 16.02%]) and 8.78% (95% CI [3.83%, 14.24%]) relative to

PT+FT, respectively; each produces six harmful alternative selections, corresponding to 7.50% of the 80 held-out cases. The screening outcome with no retained alternative therefore reverts to the adapted reference, yielding zero paired difference. All three pipelines satisfy both deployment limits in every case.

5.4. Deployment Feasibility and Cost

Feasibility in this subsection is assessed using the operation-count and parameter-count limits defined in Section 3.2. Figure 8 shows how the three deployment-limit settings affect candidate feasibility and final model selection.



(a) Candidate filtering

(b) Final model selection

Figure 8: Effects of the three deployment-limit settings on (a) the number of feasible candidates and (b) the selected model configurations.

Under the tight setting, four of the seven candidates in the main candidate bank are deployment-feasible, whereas all seven are feasible under the medium and loose settings.

For the main candidate bank under the tight setting, a single-limit analysis shows that the reduction is caused by the parameter-count limit: all seven candidates satisfy the operation-count limit, while the parameter-count limit excludes three MLP candidates.

To test whether the two limits can independently restrict candidate feasibility, we form an eight-candidate stress bank by adding one configuration from the predefined architecture space and use diagnostic thresholds derived from the resulting model-complexity profiles. For both $H = 1$ and $H = 4$, each single-limit feasible set retains seven of eight candidates, whereas the joint feasible set retains six.

Under the joint stress limits, all models selected from the expanded bank on the same 80 held-out cases satisfy both limits. Separately, for the main candidate bank, mean paired MSE reductions relative to PT+FT remain positive under the tight, medium, and loose settings: 11.35%, 15.30%, and 18.22%, respectively.

Table 3 compares target-side cost and selected-model complexity for PT+FT, the target-adaptation architecture-search baselines, and MSA-DTI.

Table 3: Target-side cost and selected-model complexity.

Method	Time per case (s)	Adapted models per case	Parameter count	Estimated operation count
PT+FT	0.204 ± 0.004	1.00	5,746	1,050,784
Few-shot NAS	13.471 ± 0.173	12.00	30,995	1,468,894
H-Meta-NAS	20.349 ± 5.722	12.00	26,627	1,700,279
Zero-shot NAS + fine-tuning	17.601 ± 0.150	12.00	62,076	1,571,621
MSA-DTI	5.676 ± 0.059	6.25	14,892	751,433

Time is mean $\pm$ standard deviation over five runs. Parameter counts and estimated operation counts are averaged over the final models selected in the held-out cases.

Relative to PT+FT, MSA-DTI incurs higher target-side instantiation time while achieving lower held-out MSE in the main evaluation. It is 2.37× faster than few-shot NAS and 3.10× faster than zero-shot NAS with fine-tuning. The reported target-side times are measured for one instantiation case on one GPU; concurrent-case scaling and platform-level scheduling are outside the present evaluation.

Hardware profiling evaluates every executable architecture on an Intel Core i7-12700H CPU and an NVIDIA RTX 3060 Laptop GPU. Median batch-size-one inference latency ranges from 0.093 to 1.980 ms on the CPU and from 0.133 to 4.170 ms on the GPU. Serialized model size ranges from 10.3 to 315.9 KiB. Operation count is positively associated

with median latency, whereas parameter count preserves the serialized-size ranking in all measured settings. Peak GPU allocation is not monotonic in parameter count. These measurements support the two counts as distinct model-level complexity indicators; their hardware-level implications remain host-dependent.

5.5. Component Analysis

Table 4 reports component variants on an independent diagnostic pool. The reference-only and two-reference-initialization variants do not reproduce the cross-architecture gain. Without the relative margin, diagnostic MSE is nearly unchanged, but alternatives are selected more often and the harmful-selection rate increases from 15.00% to 17.50%. The corresponding confidence intervals overlap, so this increase is treated as an observed rather than statistically established difference. Removing deployment filtering slightly lowers MSE but reduces limit satisfaction from 100% to 96.25%.

Overall, the main held-out result evaluates the complete MSA-DTI pipeline, while the diagnostic controls distinguish the roles of individual design choices. The cross-architecture gain remains sensitive to source training. The relative margin mainly affects replacement conservativeness, whereas deployment filtering enforces limit satisfaction.

Table 4: Component variants on the independent diagnostic pool.

Variant	MSE↓	Worst-10%↓	Paired MSE reduction (%)↑	Harmful-selection rate (%)↓	Limit satisfaction (%)↑
MSA-DTI	0.004063	0.015764	15.79	15.00	100.00
Earlier source training	0.004876	0.019044	1.75	2.50	100.00
Reference only	0.005051	0.019512	0.00	0.00	100.00
Two reference initializations	0.004917	0.019136	1.29	1.25	100.00
Without relative margin	0.004055	0.015690	16.16	17.50	100.00
Without deployment filtering	0.004037	0.015635	16.13	13.75	96.25

Paired MSE reduction is computed relative to the adapted reference on the same diagnostic pool; harmful-selection rate is computed over all diagnostic cases. The earlier source-training row changes the full source-training procedure for the alternatives, so it measures candidate-bank sensitivity rather than a single-component ablation.

5.6. External Transfer Evaluation

We also evaluate MSA-DTI on Alibaba Cluster Trace v2018 [32]. The Alibaba data are divided into 20 source, 20 calibration, and 40 held-out machines, giving 80 calibration and 160 held-out cases. Source models are retrained on the Alibaba source pool, while the architecture definitions, deployment filtering rule, support-size settings, target-adaptation procedure, and margin grid remain unchanged. Each Alibaba case uses 80 validation, 60 check, and 200 test windows. The workload observations come from the production trace, while the deployment-limit tiers use the same controlled model-complexity settings as in the main study.

When the main-study margin $\tau = 0.10$, fixed before the Alibaba evaluation, is transferred without recalibration, the mean paired MSE reduction is 4.20%, with a 95% confidence interval of $[2.03\%, 6.70\%]$. The harmful-selection rate is 6.25%, exceeding the original 5% calibration criterion. No margin in the preset grid satisfies the calibration criteria on the Alibaba calibration pool.

Under the calibration rule, the adapted reference is therefore retained. The fixed $\tau = 0.10$ result is reported separately to assess direct transfer of the main-study margin.

6. Conclusion

This paper presented MSA-DTI for DT model instantiation with few-shot target adaptation under per-model operation-count and parameter-count limits. MSA-DTI filters infeasible candidates from the source-trained bank, adapts the remaining candidates under a common update budget, and applies a calibrated reference-model replacement rule based on post-adaptation validation performance. In the main held-out evaluation, MSA-DTI reduces MSE by 14.60% on average across paired cases relative to PT+FT while satisfying both deployment limits in all cases. The confidence interval for the held-out harmful-selection rate spans the 5% calibration criterion, and candidate discovery shows sensitivity to the screening sample. In a separate end-to-end screening analysis, the two nonempty screening-derived banks yield positive mean paired MSE reductions relative to PT+FT with confidence intervals above zero, while the no-alternative screening outcome reverts to the adapted reference. On Alibaba, direct transfer of the fixed main-study margin yields a positive mean paired MSE reduction, but no margin in the preset grid satisfies the calibration criteria. The current evaluation is limited to a controlled synthetic benchmark and one external workload trace. Future

work includes investigating more stable candidate-discovery procedures, uncertainty-aware selection, and deployment limits derived from measurements on additional platforms.

CRediT Authorship Contribution Statement

The final author contribution statement will be added after the author list is confirmed.

Funding

Funding information will be added after confirmation.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

The source code, frozen configurations, processed results, plot-ready data, and reproduction scripts underlying the reported results are available in GitHub Release v1.2.7: https://github.com/w924871681/paper-code-DT-Twin/releases/tag/v1.2.7. The original Alibaba Cluster Trace v2018 is not redistributed and remains available from its official repository: https://github.com/alibaba/clusterdata. The released documentation specifies preprocessing, disjoint data splits, margin handling, and held-out evaluation.

Declaration of Generative AI and AI-Assisted Technologies in the Manuscript Preparation Process

During the preparation of this work, the authors used OpenAI ChatGPT to assist with language refinement, manuscript organization, and readability improvement. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

[1] L. U. Khan, Z. Han, W. Saad, E. Hossain, M. Guizani, and C. S. Hong, "Digital twin of wireless systems: Overview, taxonomy, challenges, and opportunities," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 4, pp. 2230–2254, Fourth Quarter 2022, doi: 10.1109/COMST.2022.3198273.

[2] S. Mihai, M. Yaqoob, D. V. Hung, W. Davis, P. Towakel, M. Raza, M. Karamanoglu, B. Barn, D. Shetve, R. V. Prasad, H. Venkataraman, R. Trestian, and H. X. Nguyen, "Digital twins: A survey on enabling technologies, challenges, trends and future prospects," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 4, pp. 2255–2291, Fourth Quarter 2022, doi: 10.1109/COMST.2022.3208773.

[3] H. Xu, J. Wu, Q. Pan, X. Guan, and M. Guizani, "A survey on digital twin for Industrial Internet of Things: Applications, technologies and tools," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 4, pp. 2569–2598, Fourth Quarter 2023, doi: 10.1109/COMST.2023.3297395.

[4] J. Li, S. Guo, W. Liang, J. Wang, Q. Chen, Y. Zeng, B. Ye, and X. Jia, "Digital twin-enabled service provisioning in edge computing via continual learning," *IEEE Transactions on Mobile Computing*, vol. 23, no. 6, pp. 7335–7350, 2024, doi: 10.1109/TMC.2023.3332668.

[5] H. Sartaj, S. Ali, and J. M. Gjøby, "MeDeT: Medical device digital twins creation with few-shot meta-learning," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 6, Art. no. 158, pp. 1–36, 2025, doi: 10.1145/3708534.

[6] A. Vettoruzzo, M.-R. Bouguelia, J. Vanschoren, T. Rögnvaldsson, and K. C. Santosh, "Advances and challenges in meta-learning: A technical review," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 7, pp. 4763–4779, 2024, doi: 10.1109/TPAMI.2024.3357847.

[7] H. Gharoun, F. Momenifar, F. Chen, and A. H. Gandomi, "Meta-learning approaches for few-shot learning: A survey of recent advances," *ACM Computing Surveys*, vol. 56, no. 12, Art. no. 294, pp. 1–41, 2024, doi: 10.1145/3659943.

[8] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, and Q. V. Le, "MnasNet: Platform-aware neural architecture search for mobile," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 2820–2828, doi: 10.1109/CVPR.2019.00293.

[9] B. Wu, X. Dai, P. Zhang, Y. Wang, F. Sun, Y. Wu, Y. Tian, P. Vajda, Y. Jia, and K. Keutzer, "FBNet: Hardware-aware efficient ConvNet design via differentiable neural architecture search," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 10734–10742, doi: 10.1109/CVPR.2019.01099.

[10] Y. Zhao, L. Wang, and T. Guo, "Multi-objective neural architecture search by learning search space partitions," *Journal of Machine Learning Research*, vol. 25, no. 177, pp. 1–41, 2024.

[11] Y. Zhao, X. Gao, I. Shumailov, N. Fusi, and R. Mullins, "Rapid model architecture adaption for meta-learning," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, doi: 10.52202/068431-1360.

[12] D. Lian, Y. Zheng, Y. Xu, Y. Lu, L. Lin, P. Zhao, J. Huang, and S. Gao, "Towards fast adaptation of neural architectures with meta learning," in *Proc. International Conference on Learning Representations (ICLR)*, 2020.

[13] P. Eustratiadis, L. Dudziak, D. Li, and T. Hospedales, "Neural fine-tuning search for few-shot learning," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[14] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 4, pp. 2405–2415, 2019, doi: 10.1109/TII.2018.2873186.

[15] B. Qin, H. Pan, Y. Dai, X. Si, X. Huang, C. Yuen, and Y. Zhang, "Machine and deep learning for digital twin networks: A survey," *IEEE Internet of Things Journal*, vol. 11, no. 21, pp. 34694–34716, Nov. 2024, doi: 10.1109/JIOT.2024.3416733.

[16] A. Hakiri, A. Gokhale, S. Ben Yahia, and N. Mellouli, "A comprehensive survey on digital twin for future networks and emerging Internet of Things industry," *Computer Networks*, vol. 244, Art. no. 110350, 2024, doi: 10.1016/j.comnet.2024.110350.

[17] Y. Wu, K. Zhang, and Y. Zhang, "Digital twin networks: A survey," *IEEE Internet of Things Journal*, vol. 8, no. 18, pp. 13789–13804, Sept. 2021, doi: 10.1109/JIOT.2021.3079510.

[18] M. Emami Khansari and S. Sharifian, "A deep reinforcement learning approach towards distributed Function as a Service (FaaS) based edge application orchestration in cloud-edge continuum," *Journal of Network and Computer Applications*, vol. 233, Art. no. 104042, 2025, doi: 10.1016/j.jnca.2024.104042.

[19] D. Hortelano, I. de Miguel, R. J. Durán Barroso, J. C. Aguado, N. Merayo, L. Ruiz, A. Asensio, X. Masip-Bruin, P. Fernández, R. M. Lorenzo, and E. J. Abril, "A comprehensive survey on reinforcement-learning-based computation offloading techniques in Edge Computing Systems," *Journal of Network and Computer Applications*, vol. 216, Art. no. 103669, 2023, doi: 10.1016/j.jnca.2023.103669.

[20] J. Son, S. Lee, and G. Kim, "When meta-learning meets online and continual learning: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, no. 1, pp. 413–432, 2025, doi: 10.1109/TPAMI.2024.3463709.

[21] L. M. Zintgraf, K. Shiarlis, V. Kurin, K. Hofmann, and S. Whiteson, "Fast context adaptation via meta-learning," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, vol. 97 of *Proc. Mach. Learn. Res.*, pp. 7693–7702, 2019.

[22] Y. Zhao, T. Zhang, J. Li, and Y. Tian, "Dual adaptive representation alignment for cross-domain few-shot learning," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 10, pp. 11720–11732, 2023, doi: 10.1109/TPAMI.2023.3272697.

[23] J. Wu, X. Liu, X. Yin, T. Zhang, and Y. Zhang, "Task-adaptive prompted transformer for cross-domain few-shot learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 6, pp. 6012–6020, 2024, doi: 10.1609/aaai.v38i6.28416.

[24] W. Wang, L. Duan, Y. Wang, J. Fan, and Z. Zhang, "MMT: Cross domain few-shot learning via meta-memory transfer," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 12, pp. 15018–15035, Dec. 2023, doi: 10.1109/TPAMI.2023.3306352.

[25] Z. Zuo, Y. Huang, Z. Li, Y. Jiang, and C. Liu, "Mixed contrastive transfer learning for few-shot workload prediction in the cloud," *Computing*, vol. 107, no. 1, Art. no. 5, 2025, doi: 10.1007/s00607-024-01366-y.

[26] Y. Zhao, L. Wang, Y. Tian, R. Fonseca, and T. Guo, "Few-shot neural architecture search," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, vol. 139 of *Proc. Mach. Learn. Res.*, pp. 12707–12718, 2021.

[27] G. Li, D. Hoang, K. Bhardwaj, M. Lin, Z. Wang, and R. Marculescu, "Zero-shot neural architecture search: Challenges, solutions, and opportunities," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 7618–7635, Dec. 2024, doi: 10.1109/TPAMI.2024.3395423.

[28] M.-T. Wu, H.-I. Lin, and C.-W. Tsai, "A training-free neural architecture search algorithm based on search economics," *IEEE Transactions on Evolutionary Computation*, vol. 28, no. 2, pp. 445–459, Apr. 2024, doi: 10.1109/TEVC.2023.3264533.

[29] Y.-M. Kim, S. Song, B.-M. Koo, J. Son, Y. Lee, and J.-G. Baek, "Enhancing long-term cloud workload forecasting framework: Anomaly handling and ensemble learning in multivariate time series," *IEEE Transactions on Cloud Computing*, vol. 12, no. 2, pp. 789–799, Apr.–Jun. 2024, doi: 10.1109/TCC.2024.3400859.

[30] F. Zhao, W. Lin, S. Lin, S. Tang, and K. Li, "MSCNet: Multi-scale network with convolutions for long-term cloud workload prediction," *IEEE Transactions on Services Computing*, vol. 18, no. 2, pp. 969–982, Mar.–Apr. 2025, doi: 10.1109/TSC.2025.3536313.

[31] B. Feng and Z. Ding, "Application-oriented cloud workload prediction: A survey and new perspectives," *Tsinghua Science and Technology*, vol. 30, no. 1, pp. 34–54, Feb. 2025, doi: 10.26599/TST.2024.9010024.

[32] [dataset] Alibaba Group, "Alibaba Cluster Trace Program: Cluster Trace v2018," GitHub repository, 2018. [Online]. Available: `https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2018`. Accessed: Jul. 3, 2026.

[33] J. Bornschein, F. Visin, and S. Osindero, "Small data, big decisions: Model selection in the small-data regime," in *Proc. 37th Int. Conf. Mach. Learn. (ICML)*, vol. 119 of *Proc. Mach. Learn. Res.*, pp. 1035–1044, 2020.

[34] A. Lacoste, M. Marchand, F. Laviolette, and H. Larochelle, "Agnostic Bayesian learning of ensembles," in *Proc. 31st Int. Conf. Mach. Learn. (ICML)*, vol. 32 of *Proc. Mach. Learn. Res.*, pp. 611–619, 2014.